

M6 – Seguridad en Librerías de Terceros y Auditoría

Objetivo: Aprender la importancia de auditar código y dependencias externas para evitar vulnerabilidades en aplicaciones web.

José Picón

SEGURIDAD EN LIBRERÍAS DE TERCEROS Y AUDITORÍA

Introducción a la Seguridad en Librerías de Terceros

¿Qué son las librerías de terceros?

- Código externo utilizado en aplicaciones (*frameworks, APIs, plugins*).
- Ejemplos: **jQuery, Bootstrap, OpenSSL, Log4j**.



Riesgos de usar librerías de terceros

- Dependencias desactualizadas con vulnerabilidades conocidas.
- Código malicioso oculto en paquetes externos.
- Uso de librerías sin mantenimiento ni actualizaciones.

Buenas prácticas en la gestión de librerías

- Mantener dependencias **actualizadas y verificadas**.
- Usar **repositorios oficiales** y evitar paquetes no auditados.
- Realizar análisis de seguridad con herramientas automatizadas.

🚩 CVE-2022-31160 Detail

MODIFIED

This CVE record has been updated after NVD enrichment efforts were completed. Enrichment data supplied by the NVD may require amendment due to these changes.

Description

jQuery UI is a curated set of user interface interactions, effects, widgets, and themes built on top of jQuery. Versions prior to 1.13.2 are potentially vulnerable to cross-site scripting. Initializing a checkboxradio widget on an input enclosed within a label makes that parent label contents considered as the input label. Calling ``.checkboxradio("refresh")`` on such a widget and the initial HTML contained encoded HTML entities will make them erroneously get decoded. This can lead to potentially executing JavaScript code. The bug has been patched in jQuery UI 1.13.2. To remediate the issue, someone who can change the initial HTML can wrap all the non-input contents of the ``label`` in a ``span``.

SAST (STATIC APPLICATION SECURITY TESTING)

¿Qué es SAST?

Definición

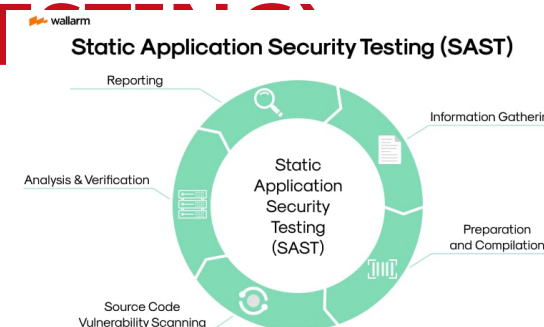
- 0 Técnica de análisis de seguridad **en el código fuente** sin ejecutarlo.
- 0 Detecta vulnerabilidades como **inyección SQL, XSS y uso inseguro de funciones**.

Ventajas

- 0 Identifica problemas antes de la ejecución.
- 0 Se integra en CI/CD para análisis automático (*es un conjunto de prácticas de desarrollo de software que permite la integración automática, pruebas y despliegue de código de manera rápida y segura*)
- 0 Permite auditoría temprana en el desarrollo.

Ejemplo de herramienta SAST

- 0 SonarQube, Checkmarx, Semgrep.



<https://www.wallarm.com/what/sast-dast-iast-and-rasp>

sonarqube

Checkmarx

semgrep

SAST (STATIC APPLICATION SECURITY TESTING)

Ejemplo de Análisis con SAST (SonarQube)

Ejemplo de código vulnerable

```
$input = $_GET['user'];  
$query = "SELECT * FROM users WHERE username = '$input'";
```

- SonarQube detecta inyección SQL en \$input.

Solución recomendada

```
$stmt = $db->prepare("SELECT * FROM users WHERE username = ?");  
$stmt->execute([$input]);
```

- Uso de **consultas parametrizadas** para evitar SQL Injection.



ACTIVIDAD

M6-A1-Análisis de Código Estático con SAST (Static Application Security Testing)

Tema: Identificación de vulnerabilidades en código fuente

Objetivo: Usar SAST para detectar problemas de seguridad en código sin ejecutarlo

DAST (DYNAMIC APPLICATION SECURITY TESTING)

¿Qué es DAST?

Definición

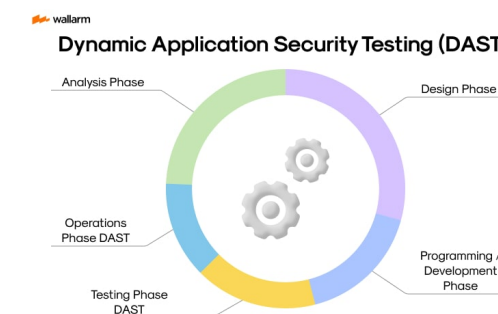
- Técnica de análisis de seguridad **durante la ejecución de la aplicación**.
- Simula ataques reales como **inyección SQL, XSS y CSRF**.

Ventajas

- Identifica vulnerabilidades explotables en tiempo real.
- No requiere acceso al código fuente.
- Evalúa seguridad en entornos productivos.

Ejemplo de herramienta DAST

- OWASP ZAP, Burp Suite, Acunetix.



<https://www.wallarm.com/what/sast-dast-iaast-and-rasp>



OWASP
Zed Attack Proxy



DAST (DYNAMIC APPLICATION SECURITY TESTING)

Ejemplo de Análisis con DAST (OWASP ZAP)

Configuración de escaneo

- Se ejecuta OWASP ZAP y se configura el escaneo contra un sitio web.
- OWASP ZAP detecta URLs vulnerables a inyección SQL y XSS.

Ejemplo de resultado de escaneo

Alerta: Inyección SQL detectada en <http://example.com/login>

Riesgo: Alto

Solución: Usar consultas parametrizadas

- Proporciona recomendaciones de mitigación.



OWASP
Zed Attack Proxy

ACTIVIDAD

M6-A2-Análisis de Seguridad Dinámico con DAST (Dynamic Application Security Testing)

Tema: Escaneo de vulnerabilidades en aplicaciones en ejecución

Objetivo: Usar DAST para detectar vulnerabilidades en una aplicación en ejecución

DEPENDENCY-CHECK: ANÁLISIS DE DEPENDENCIAS

¿Qué es Dependency-Check?



Definición

- Herramienta de OWASP que analiza **dependencias de proyectos** en busca de vulnerabilidades conocidas.

Funcionamiento

- Escanea paquetes en **Java, Python, JavaScript, .NET, PHP**.
- Compara versiones con la base de datos de vulnerabilidades **CVE (Common Vulnerabilities and Exposures)**.

Ejemplo de uso en un proyecto Java (Maven)

```
<dependency>  
  <groupId>org.apache.logging.log4j</groupId>  
  <artifactId>log4j-core</artifactId>  
  <version>2.14.1</version>  
</dependency>
```

- **Dependency-Check alerta si la versión es vulnerable** (*Ejemplo: Log4Shell en Log4j*).

LOGGING & MONITORING

Ejemplo de Análisis con OWASP Dependency-Check

Ejecución en un proyecto

- Comando en línea para analizar dependencias en un proyecto Node.js:
`dependency-check --project "MiApp" --scan ./node_modules`
- Genera un **reporte/informe HTML** con vulnerabilidades detectadas.

Ejemplo de resultado

Paquete vulnerable: lodash 4.17.15

CVE-2021-23337 - Severidad: Alta

Solución: Actualizar a lodash 4.17.21

- **Recomendación:** Actualizar dependencias afectadas.



ACTIVIDAD

M6-A3-Análisis de Dependencias con Dependency-Check

Tema: Detección de librerías y paquetes vulnerables

Objetivo: Usar Dependency-Check para analizar dependencias en un proyecto

OWASP ZAP: ESCANEO DE SEGURIDAD EN APLICACIONES WEB

¿Qué es OWASP ZAP?

Definición

- Herramienta gratuita de **escaneo de seguridad en aplicaciones web**.
- Permite **pruebas de penetración automatizadas e interactivas**.

Principales características

- Escaneo automático de vulnerabilidades.
- Proxy para interceptar tráfico web.
- Fuzzing y pruebas manuales de inyección.



OWASP
Zed Attack Proxy

OWASP ZAP: ESCANEO DE SEGURIDAD EN APLICACIONES WEB

Uso Básico de OWASP ZAP

Configuración inicial

- 0 Ejecutar ZAP y configurar la URL de prueba.
- 0 Iniciar escaneo pasivo y activo.

Ejemplo de resultado

Vulnerabilidad detectada: XSS reflejado en /search?q=<script>

Severidad: Media

Solución recomendada: Escapar y validar entrada del usuario

- 0 Informa los riesgos y soluciones recomendadas.



OWASP
Zed Attack Proxy

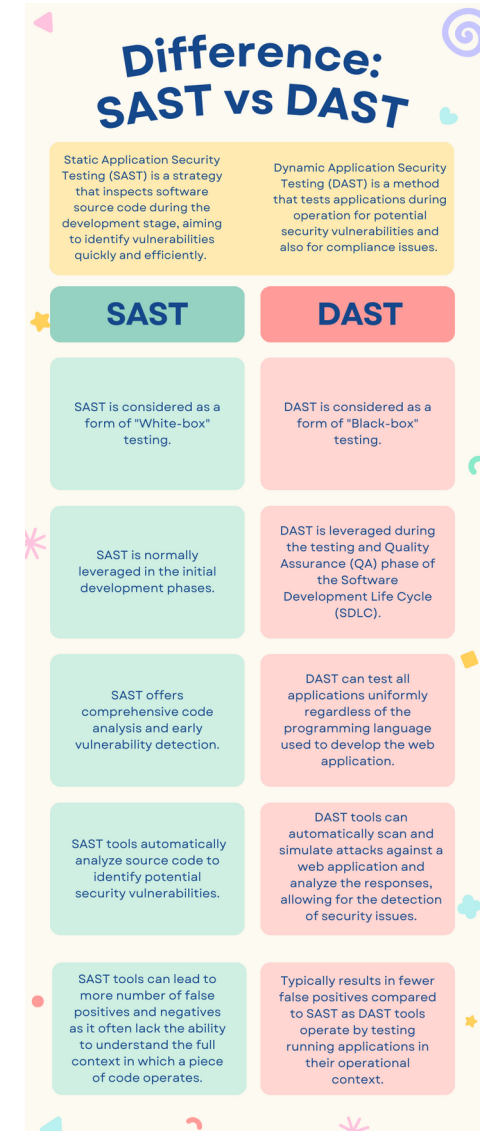
SAST vs. DAST

Comparación SAST vs. DAST

- **SAST** es útil para detectar vulnerabilidades en el código fuente.
- **DAST** evalúa la seguridad de la aplicación en tiempo real.

Característica	SAST	DAST
Tipo de análisis	Estático	Dinámico
Requiere acceso al código	Sí	No
Detecta vulnerabilidades antes de la ejecución	Sí	No
Identifica fallos de configuración	No	Sí

<https://www.balbix.com/insights/sast-dast-iaast-application-security-appsec-testing-tools/>



ACTIVIDAD

M6-A4-Escaneo de Seguridad con OWASP ZAP

Tema: Identificación de vulnerabilidades con OWASP ZAP

Objetivo: Usar OWASP ZAP para encontrar y corregir vulnerabilidades en una aplicación